

# Exploration Report: $\alpha, \beta$ -CROWN Models and Configuration System

Jaemin Paek

This report surveys the  $\alpha, \beta$ -CROWN repository’s built-in models directory and configuration system. All figures were obtained by inspecting the cloned repository at `complete_verifier/` and cross-checked against the actual files (two independent counts per number).

## 1 What models are provided (architecture, format)

The built-in models live under `complete_verifier/models/`, in 13 category directories (`bab_attack`, `cifar10_resnet`, `crown-ibp`, `custom_op`, `custom_specs`, `eran`, `l2_norm`, `marabou_cifar10`, `non_relu`, `oval`, `sdp`, `toy`, `vnncomp22`).

Models are stored as **PyTorch checkpoints**, not ONNX:

| Extension           | Count    | Meaning                                     |
|---------------------|----------|---------------------------------------------|
| <code>.pth</code>   | 24       | PyTorch <code>state_dict</code> checkpoints |
| <code>.model</code> | 8        | PyTorch checkpoints (SDP naming)            |
| <code>.pkl</code>   | 4        | pickled weights                             |
| <code>.pt</code>    | 1        | PyTorch checkpoint                          |
| <code>.onnx</code>  | <b>0</b> | (none in <code>models/</code> )             |

There are **no ONNX files** in the directory. The checkpoints hold only weights; the matching architectures are defined as  $\sim 97$  constructor functions in `model_defs.py`, and a config selects one by `name`: and loads weights from `path`:. ONNX is supported for *external* models (via `onnx_path`:), but the bundled collection is PyTorch-based. By name keyword the architecture mix is approximately 35 ResNet, 29 CNN/conv, and 21 fully-connected/MLP networks, with no recurrent models — i.e., small-to-medium CIFAR-10 and MNIST image classifiers drawn from verification benchmarks (ERAN, SDP, OVAL, CROWN-IBP, VNN-COMP).

## 2 What verification configurations are available

$\alpha, \beta$ -CROWN is driven entirely by YAML. `exp_configs/` contains **263 YAML files** across 10 directories:

| Directory         | Files | Directory  | Files |
|-------------------|-------|------------|-------|
| BICCOS            | 128   | GCP-CROWN  | 13    |
| beta_crown        | 27    | vnncomp23  | 13    |
| vnncomp22         | 27    | vnncomp25  | 9     |
| tutorial_examples | 18    | vnncomp24  | 8     |
| vnncomp21         | 14    | bab_attack | 6     |

Configs are organized by competition year (vnncomp21–25) and by algorithmic method (beta\_crown, GCP-CROWN, BICCOS), plus a tutorial set. A config has **9 top-level sections** (verified independently from `arguments.py`): `general`, `model`, `data`, `specification`, `solver`, `bab`, `attack`, `debug`, `solving`, `exposing` **308 options**. The most relevant keys are `model` (`name+path` or `onnx_path`, `input_shape`); `data` (`dataset`, `mean/std`, `start/end`); `specification` (`norm`, `epsilon`, or `vnnlib_path`); `solver` (`batch_size`,  $\alpha/\beta$ -CROWN rates); `bab` (`timeout`, `branching.method`  $\in \{\text{babsr, fsb, kfsb}\}$ ); and `attack` (PGD settings). The property is almost always an  $\ell_\infty$  robustness ball: of configs that set a norm, 140 use `.inf` and 2 use `L2`. Datasets are mostly CIFAR-10 and MNIST variants (e.g. `CIFAR_SDP`, `MNIST_SDP`, `_ERAN` normalization variants, `CIFAR100`, `Customized`).

### 3 How $\alpha, \beta$ -CROWN’s model specification differs from Marabou’s

| Aspect          | $\alpha, \beta$ -CROWN                            | Marabou                                  |
|-----------------|---------------------------------------------------|------------------------------------------|
| Interface       | declarative YAML config                           | imperative Python API (maraboupy)        |
| Model load      | name+path (113 configs) or onnx_path (4)          | Marabou.read_onnx(path)                  |
| Input region    | epsilon over dataset, or vnnlib_path              | per-variable setLowerBound/setUpperBound |
| Output property | dataset auto-gen, or VNNLIB                       | hand-built MarabouUtils.Equation         |
| Batch runs      | one CSV $\rightarrow$ many instances, one process | one script per query                     |
| Solver control  | config keys (branching, iterations, PGD)          | a few API options                        |

$\alpha, \beta$ -CROWN separates *what to verify* (declarative config / VNNLIB) from the code, so a new experiment is a new YAML file. Marabou builds the query *programmatically*: the model is read with `read_onnx`, the input box is set variable by variable, and the output property is assembled from `Equation` objects in Python.

### 4 Organization compared with Marabou’s resources

$\alpha, \beta$ -CROWN ships a large, curated benchmark suite — a models directory grouped by source plus 263 ready-to-run configs organized by competition year and method. Marabou is closer to a verification *library*: it provides the API and a few example networks, and the user writes the query script. The trade-off is convenience and reproducibility ( $\alpha, \beta$ -CROWN’s config/VNNLIB ecosystem) versus flexibility (Marabou’s direct programmatic control).